Agentic Text Comparator

System Architecture, Cognitive Reasoning Models, Memory Subsystems & Production Resilience

Author / Engineer: Shiva	**Runtime Target:** Python 3.10+ (Zero-Dependency)
Architecture: ReAct + Reflexion Hybrid	**Storage:** SQLite3 Persistent Semantic Memory
Status: Production Verified	**Date:** August 2026

Executive Summary

Modern enterprise workflows in legal compliance, financial reconciliation, regulatory auditing, and engineering change-control require comparing complex text documents. Standard lexical tools (such as diff or git diff) only report character or line-level edits without understanding semantic implications. Conversely, naive Large Language Model (LLM) prompts suffer from hallucinations, ungrounded outputs, lack of cross-session adaptation, and high token costs. This system implements an autonomous, tool-grounded **ReAct + Reflexion** agent equipped with a hybrid semantic/lexical SQLite3 persistent memory, a fault-tolerant execution harness, and a real-time visual inspection studio.

1. Use Case & Design Rationale

1.1 Target Problem Space

The system is purpose-built for high-consequence document comparison scenarios where subtle modifications carry outsized impact:

- **Financial & Earnings Disclosures:** Spotting modified forecasts, revised EBITDA targets, team allocations, or cloud expenditures across quarters.
- **Legal & Compliance Contracts:** Identifying escalated liability clauses, altered dispute windows (e.g. 60 days mediation → 15 days binding arbitration), and newly imposed penalties.
- **Engineering Service Level Agreements (SLAs):** Tracking uptime modifications (99.5% → 99.99%) and response window tightenings.
- **Semantic & Sentiment Shifts:** Detecting severe tone flips from positive/neutral statements to negative, risk-laden statements (e.g., 'good' → 'bad', 'stable' → 'unstable').

1.2 Why Traditional Approaches Fail

Approach	Strengths	Critical Flaws in Enterprise Production
Pure Lexical Diff (ndiff, diff)	100% deterministic, instant execution.	Cannot classify severity; blind to sentiment, omissions, or semantic meaning.
Direct LLM Prompting (One-shot CoT)	Capable of summarization and tone analysis.	Hallucinates line diffs; token-wasteful; lacks session memory; ungrounded.
Our Agentic System (ReAct + Reflexion)	Tool-grounded accuracy + semantic awareness + persistent cross-session memory.	Requires structured orchestration harness (implemented in zero-dependency Python).

2. Agentic Reasoning Patterns Applied & Why

2.1 The ReAct (Reason + Act) Paradigm

1. PERCEIVE: Ingests conversation history, user input texts (Text A & Text B), tool schemas, and injected memory preferences from SQLite3.

2. REASON: Dispatches context to the LLM to determine the next optimal action (e.g. compute diff vs categorize vs generate final report).

3. ACT: Executes deterministic Python tools (compute_text_diff or categorize_discrepancy) and captures exact stdout/JSON observations.

4. REFLECT: The reflect() function inspects the tool outcome, checks goal completion, and decides whether to continue or terminate.

```
# ReAct Execution Loop in agent_loop.py
while not state["done"] and state["iterations"] < max_iterations:
    # 1 & 2: Perceive context & Reason next step via LLM
    decision = reason(state["messages"], TOOL_SCHEMAS, client, model)
    state["messages"].append(assistant_message_dict)

    # 3: Act (dispatch external tool or capture terminal text)
    tool_name, tool_call_id, action_result = act(decision, TOOL_HANDLERS)

    # 4: Reflect & Check Terminal State
    if reflect(state, final_content=decision.content, action_result=action_result):
        break  # Goal satisfied; clean exit
```

2.2 The Reflexion Pattern (Dual-Tiered)

• **Tier 1 (Intra-Session Goal Validation):** Evaluates after each tool execution whether the extracted discrepancy data is sufficient for final synthesis. If unrecoverable errors occur, it safely transitions state to abort without infinite cycling.

• **Tier 2 (Inter-Session Episodic Rule Injection):** When a user sets an operational preference in Session 1 (e.g., *'Focus strictly on numerical budget changes; suppress all stylistic shifts'*), this rule is permanently persisted in SQLite3 and automatically recalled during Session 2 to constrain future comparison outputs.

2.3 Why Alternative Agentic Patterns Were Rejected

Pattern	Evaluation & Architectural Verdict

3. Memory Architecture & Concrete Implementation

3.1 Tripartite Memory Topology

- **Short-Term Memory:** The active conversation array state['messages'] holding exact tool calls, tool responses, and intermediate observations within the single comparison run.
- **Episodic Working Memory:** Relevant recalled preference rules dynamically formatted into a system-prompt injection block (--- RECALLED CONTEXT ---).
- **Long-Term Persistent Memory:** SQLite3 database (agent_memory.db) with full schema versioning, storing serialized text embeddings, lexical tokens, match frequency, and timestamps.

3.2 Concrete SQLite3 Database Schema (DDL)

```
CREATE TABLE IF NOT EXISTS agent_memory (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id TEXT NOT NULL,
    memory_key TEXT NOT NULL,
    memory_value TEXT NOT NULL,
    embedding_blob TEXT,       -- Serialized float array for semantic retrieval
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_accessed TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    access_count INTEGER DEFAULT 1
);
CREATE INDEX IF NOT EXISTS idx_session_key ON agent_memory(session_id, memory_key);
```

3.3 Hybrid Scoring Mathematical Formulation

Key Takeaway: Hybrid Score = 0.7 × CosineSimilarity(Q_emb, Mem_emb) + 0.3 × LexicalRatio(Q_text, Mem_text)
Threshold: Memories with Hybrid Score ≥ 0.35 are retrieved, ranked by score descending, and injected into the cognitive loop.

3.4 End-to-End Memory Lifecycle Walkthrough

4. Production Harness Resilience & Failure Modes Defended

Failure Mode	Root Cause / Risk	Harness Defense Mechanism

4.1 Structured JSON Observability

```
{"timestamp": "2026-08-27T05:30:13Z", "level": "INFO", "logger": "agent_harness",
 "event_type": "iteration_step", "payload": {"session_id": "session_finance_dept",
 "iteration": 2, "latency_ms": 0.42, "total_tokens_spent": 460, "done": false}}
```

5. Honest Reflections: What Failed & What Was Done Differently

5.1 Failure 1: Cross-Session Memory Bleed

5.2 Failure 2: Shallow String Diffing vs. Semantic Sentiment Shift

5.3 Failure 3: Browser CORS & Protocol Blockers on Local HTML

5.4 Architectural Roadmap for Version 2.0

- **Vector Embedding Integration:** Incorporate dense vector embeddings (e.g. text-embedding-3-small or local ONNX) alongside SQLite3 for semantic similarity across large corpora.
- **Hierarchical AST Diffing:** Add specialized parsers for JSON, YAML, and Python AST trees to distinguish syntax-level changes from functional modifications.
- **Distributed Memory Clustering:** Extend the single-file SQLite3 database to a distributed Postgres/pgvector backend for multi-tenant enterprise deployment.

6. Complete System Architecture Specification

Layer / Module	Key File(s)	Core Responsibilities & Capabilities

7. Conclusion & Verification Summary

Key Takeaway: Verification Status: All 3 core evaluation scenarios (Cold-start multi-iteration, Memory recall active, Fault injection & recovery) and custom text analyses have been fully verified with 100% test pass rate across unit, integration, and UI layers.